

# Network Analysis of Food System Waste: Compounded Path Loss and a Causal Graph over Network Parameters

Michael Harrison Lee

March 2025

Revised September 2026

## Abstract

EcoAnalyst is a Python library that represents a supply chain, or a similar system of material, energy, and money flows, as a directed multigraph in which each node and edge loses part of what reaches it and passes on the rest. This paper states the model that version 3.0 of the library implements. A component that wastes a fraction  $w$  of its input and converts a fraction  $c$  of the remainder passes on  $r = (1 - w)c$ . The fraction delivered along a path is therefore the product of the  $r$  values, and the waste fraction is the sum of the losses that each component takes from what actually reaches it. Minimum-loss routing is a shortest-path problem under the weights  $-\ln r$ , which Dijkstra’s algorithm solves because the weights are non-negative. The routing does not enforce capacities. The paper also defines three semantics for the efficiency of a component and two estimates of loss cost, and it describes the Bayesian regression module of version 1.x as an associational analysis of loss drivers. The main addition is a causal graph that runs in parallel to the flow graph. Its variables follow structural equations that are linear on a link scale, and some of them are bound to parameters of the flow network. An intervention replaces an equation. The library estimates the effect of an intervention on a flow outcome by Monte Carlo sampling with shared exogenous noise, updates the equations from data with conjugate normal-inverse-gamma posteriors or a Laplace approximation, and checks the back-

door criterion to report which observed variables identify an effect. A cold-chain example with synthetic data shows the workflow.

# 1 Introduction

Food loss and waste occur at every stage of a supply chain, from harvest through processing, storage, transport, and retail [2]. A model that attributes loss to stages has to respect three facts. A stage can lose only what reaches it. Some stages transform the product as well as lose part of it. The loss rate of a stage depends on conditions such as temperature or equipment uptime. A decision about an intervention, for example backup power for a cold store, also needs an estimate of how much the intervention changes what reaches the consumer. That is a question about the effect of an action, and correlations in historical records do not answer it without further assumptions.

EcoAnalyst uses two linked graphs for these tasks. The flow graph describes what moves between which actors and what each component loses. A second graph, over variables such as ambient temperature and cooling uptime, describes why the loss parameters of the flow graph take the values they do. This paper makes the following contributions:

- a definition of path loss as a product of retained fractions, with an exact split of the input into delivered, wasted, and converted parts (Section 2);
- a proof that minimum-loss routing is a shortest-path problem under logarithmic weights, with a statement of what the routing does not model (Section 2.5);
- three explicit semantics for component efficiency (Section 2.3) and two loss-cost estimates (Section 3);
- a description of the 1.x regression module as associational analysis (Section 4);
- a causal graph bound to the parameters of the flow graph, with interventions, Bayesian updating, and a back-door identification check (Section 5), and a worked cold-chain example (Section 6).

Section 7 summarizes the implementation, and Sections 8 and 9 discuss limitations and conclude.

## 2 Flow network model

### 2.1 Graph and parameters

**Definition 1** (Flow network). A flow network is a directed multigraph  $G_p = (N, E, \text{src}, \text{tgt})$  with a finite node set  $N$ , a finite edge set  $E$ , and maps  $\text{src}, \text{tgt} : E \rightarrow N$ , together with a kind label for every node and edge and a parameter vector  $\theta$ . Several edges may join the same ordered pair of nodes. The parameters of a node  $v$  are a waste rate  $w_v \in [0, 1]$ , an efficiency  $\eta_v \in [0, 1]$ , an efficiency mode  $\mu_v$ , a capacity  $Q_v > 0$  per unit, a unit count  $n_v \geq 1$ , and a degradation rate  $\delta_v \in [0, 1]$ . The parameters of an edge  $e$  are a waste rate  $w_e \in [0, 1]$ , an efficiency  $\eta_e \in [0, 1]$ , an efficiency mode  $\mu_e$ , and a maximum rate  $M_e > 0$ . Waste rates, efficiencies, and modes are optional. An unset waste rate counts as zero.

A kind is an identifier that matches the pattern `[a-z][a-z0-9_]*`, that is, a lowercase letter followed by lowercase letters, digits, or underscores. The library defines the node kinds `producer`, `processor`, `handler`, and `consumer`, the additional node kinds `service` and `grid`, and the edge kinds `inventory`, `energy`, `currency`, `information`, `service`, `waste`, and `control`. Any other identifier is accepted as a custom kind. Kinds are labels. The library does not derive them from the graph structure and does not restrict which kinds may be connected. They enter the loss calculations only through an optional set  $F$  of edge kinds, which restricts a path to edges whose kind is in  $F$ . Figure 1 shows why  $G_p$  is a multigraph: the same two actors can be joined by an inventory edge in one direction and a currency edge in the other.

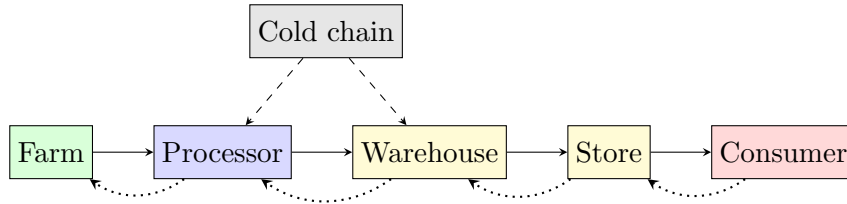


Figure 1: A food system network with parallel edges of different kinds: inventory (solid), service (dashed), and currency (dotted). A path computation with the kind filter  $F = \{\text{inventory}\}$  uses only the solid edges.

## 2.2 Retained fraction of a component

Every node and every edge is a component that acts on the quantity that reaches it. Below,  $x$  denotes a component.

**Definition 2** (Transfer). The transfer of a component  $x$  is a pair  $(\hat{w}_x, c_x) \in [0, 1]^2$ . The component wastes the fraction  $\hat{w}_x$  of its input and passes on the fraction  $c_x$  of the remainder. The rest of the remainder, which is the fraction  $(1 - \hat{w}_x)(1 - c_x)$  of the input, is the conversion shortfall. The retained fraction of  $x$  is

$$r_x = (1 - \hat{w}_x) c_x. \quad (1)$$

The effective waste rate  $\hat{w}_x$  and the conversion  $c_x$  follow from the stored parameters by the efficiency semantics of Section 2.3. In the default mode,  $c_x = 1$ ,  $\hat{w}_x = w_x$ , and  $r_x = 1 - w_x$ .

## 2.3 Efficiency semantics

Data sources report efficiency with different meanings. A mill reports the flour yield per tonne of wheat, which is a conversion ratio. A refrigerated transport operator may report the share of cargo that arrives in saleable condition, which is one minus a loss rate. EcoAnalyst makes the meaning explicit with an efficiency mode. The network has a default mode  $\mu_0$ , and a component can override it, so the mode of component  $x$  is  $m_x = \mu_x$  if  $\mu_x$  is set and  $m_x = \mu_0$  otherwise. With  $w_x$  read as zero when it is unset, the three modes are

$$\text{ignore: } \hat{w}_x = w_x, \quad c_x = 1, \quad (2)$$

$$\text{conversion: } \hat{w}_x = w_x, \quad c_x = \eta_x, \quad (3)$$

$$\text{retention: } \hat{w}_x = 1 - \eta_x, \quad c_x = 1. \quad (4)$$

In the ignore mode, which is the default, the efficiency is stored and not used. In the conversion mode the output is the input multiplied by  $(1 - w_x) \eta_x$ . The shortfall  $(1 - w_x)(1 - \eta_x)$  is reported separately and is not priced as waste. In the retention mode the efficiency is the fraction that gets through. If  $w_x$  is also set, the two values must agree, and the library raises an error when  $|1 - \eta_x - w_x| > 10^{-6}$ . If  $\eta_x$  is unset, every mode reduces to (2).

A consistency check reports retention-mode components whose two values disagree. It also warns about conversion-mode components with  $\eta_x = 1 - w_x$ .

That equality usually means that both fields describe the same loss, which the conversion mode would then count twice.

## 2.4 Path loss

A path is a sequence of nodes  $\pi = (v_0, \dots, v_k)$  in which every consecutive pair is joined by at least one edge whose kind is in  $F$ . Let  $E_F(u, v)$  be the set of edges from  $u$  to  $v$  with kind in  $F$ . Between  $v_{i-1}$  and  $v_i$  the library uses the eligible edge that passes on the largest fraction,

$$e_i = \arg \max_{e \in E_F(v_{i-1}, v_i)} r_e. \quad (5)$$

The path then has the component sequence

$$(x_1, \dots, x_{2k+1}) = (v_0, e_1, v_1, \dots, e_k, v_k),$$

which includes both end nodes. One unit of input enters  $x_1$ . Let  $q_0 = 1$  and let  $q_j = q_{j-1} r_{x_j}$  be the quantity that leaves component  $x_j$ .

**Definition 3** (Path fractions). The delivered fraction, the waste fraction, and the conversion fraction of  $\pi$  are

$$D(\pi) = q_{2k+1} = \prod_{j=1}^{2k+1} r_{x_j}, \quad (6)$$

$$W(\pi) = \sum_{j=1}^{2k+1} q_{j-1} \hat{w}_{x_j}, \quad (7)$$

$$C(\pi) = \sum_{j=1}^{2k+1} q_{j-1} (1 - \hat{w}_{x_j})(1 - c_{x_j}). \quad (8)$$

**Lemma 1** (Balance). *For every path,  $D(\pi) + W(\pi) + C(\pi) = 1$ . If no component on  $\pi$  converts, then  $W(\pi) = 1 - \prod_j (1 - \hat{w}_{x_j})$ .*

*Proof.* For each  $j$ ,

$$q_{j-1} - q_j = q_{j-1}(1 - r_{x_j}) = q_{j-1}\hat{w}_{x_j} + q_{j-1}(1 - \hat{w}_{x_j})(1 - c_{x_j}).$$

The sum over  $j$  telescopes to  $q_0 - q_{2k+1} = 1 - D(\pi) = W(\pi) + C(\pi)$ . If  $c_{x_j} = 1$  for all  $j$ , then  $C(\pi) = 0$  and  $D(\pi) = \prod_j (1 - \hat{w}_{x_j})$ .  $\square$

Lemma 1 is the mass balance of a path: every unit of input is delivered, wasted, or converted. The functions `calculate_path_transfer` and `calculate_path_waste` return these fractions together with the contribution of each component. Each term in (7) applies a loss rate to what reaches the component, not to the original input. A sum of the raw rates overstates the loss. For rates in  $[0, 1]$  the Weierstrass product inequality gives  $\prod_j (1 - w_j) \geq 1 - \sum_j w_j$ , so  $\sum_j w_j \geq 1 - \prod_j (1 - w_j)$ , with equality only when at most one rate is positive. The sum can also exceed one. For the four node rates 0.044, 0.050, 0.065, and 0.150 of Figure 2, the sum is 0.309 and the compounded loss is  $1 - (0.956)(0.950)(0.935)(0.850) = 0.278$ .

## 2.5 Minimum-loss routing

For a source  $s$  and a target  $t \neq s$ , the function `find_minimum_waste_path` returns the path that delivers the largest fraction. It first builds a simple directed graph  $H$  on  $N$ . For each ordered pair  $(u, v)$  with at least one eligible edge, let  $r^*(u, v) = \max_{e \in E_F(u, v)} r_e$ . If  $r^*(u, v) > 0$  and  $r_u > 0$ , the graph  $H$  contains the arc  $(u, v)$  with weight

$$\omega(u, v) = -\ln r^*(u, v) - \ln r_u. \quad (9)$$

An arc whose best edge or whose source node passes on nothing is left out.

**Proposition 2** (Minimum-loss routing). *Assume  $r_t > 0$ . Every  $s$ - $t$  path with  $D(\pi) > 0$  is a path in  $H$ , and every  $s$ - $t$  path  $\pi = (v_0 = s, \dots, v_k = t)$  in  $H$  satisfies*

$$\sum_{i=1}^k \omega(v_{i-1}, v_i) = -\ln D(\pi) + \ln r_t. \quad (10)$$

*The weights are non-negative. A shortest  $s$ - $t$  path in  $H$  therefore maximizes  $D(\pi)$  over all  $s$ - $t$  paths, and Dijkstra's algorithm [3] finds one. If no component converts, the same path minimizes the waste fraction  $W(\pi)$ .*

*Proof.* A path with  $D(\pi) > 0$  has  $r > 0$  for every component, so each of its arcs is in  $H$ . By (5) the edge used between  $v_{i-1}$  and  $v_i$  has  $r_{e_i} = r^*(v_{i-1}, v_i)$ . The logarithm turns the product (6) into a sum:

$$-\ln D(\pi) = -\sum_{i=0}^k \ln r_{v_i} - \sum_{i=1}^k \ln r_{e_i} = \sum_{i=1}^k (-\ln r_{e_i} - \ln r_{v_{i-1}}) - \ln r_t,$$

which is (10). The term  $\ln r_t$  is the same for every path to  $t$ , and  $-\ln$  is decreasing, so the path with the smallest weight sum has the largest  $D(\pi)$ . Every arc of  $H$  has  $r^*(u, v) \in (0, 1]$  and  $r_u \in (0, 1]$ , so  $\omega(u, v) \geq 0$ . Non-negative weights are the condition under which Dijkstra's algorithm returns a shortest path. If no component converts, Lemma 1 gives  $W(\pi) = 1 - D(\pi)$ .  $\square$

Building  $H$  takes  $O(|E|)$  time, and Dijkstra's algorithm with a binary heap takes  $O((|N| + |A|) \log |N|)$  time, where  $A$  is the arc set of  $H$  and  $|A| \leq |E|$ . The function `find_all_paths` lists up to  $K$  simple paths in order of increasing weight sum, which by (10) is the order of decreasing delivered fraction. It uses Yen's algorithm [10] as implemented in NetworkX.

**Remark 1** (What the routing does not model). When some component converts, the route maximizes the delivered fraction, and that route need not minimize waste. A path that wastes nothing but converts with  $\eta = 0.7$  delivers 0.7 with  $W = 0$ . A path with a single waste rate of 0.2 delivers 0.8 with  $W = 0.2$ . The router chooses the second path.

The weights (9) depend only on waste rates and efficiencies. Capacities  $Q_v$ , unit counts  $n_v$ , and maximum rates  $M_e$  do not enter them. The computation follows a fraction of one unit along a single path, not a quantity of flow. The result is the best single path per unit of input, and that path may be unable to carry a given supply. Allocating a supply across several paths subject to capacities, with losses on the arcs, is a generalized network flow problem [1]. The library does not solve it.

### 3 Loss cost

Both cost estimates use a price  $p(\ell)$  per unit for each node class  $\ell$ , with a default price for classes that have no price. An edge is priced by the class of its source node. Write  $\ell_v$  for the class of node  $v$ .

The capacity-based estimate treats every component on its own and assumes that it runs at full capacity:

$$L_{\text{cap}}(\theta) = \sum_{v \in N} p(\ell_v) Q_v n_v \hat{w}_v + \sum_{e \in E} p(\ell_{\text{src}(e)}) M_e \hat{w}_e. \quad (11)$$

This estimate does not follow material from one component to the next, so it is not a mass balance. It serves to rank components and to compare

two versions of a network. The hotspot ranking uses the same component quantities.

The path estimate follows an input quantity  $I$  along a path  $\pi$ . By default  $I = Q_{v_0} n_{v_0}$ . Component  $x_j$  receives  $I q_{j-1}$  and wastes  $I q_{j-1} \hat{w}_{x_j}$ , so

$$L_{\text{path}}(\pi) = I \sum_{j=1}^{2k+1} p_j q_{j-1} \hat{w}_{x_j}, \quad (12)$$

where  $p_j$  is the price of the class of  $x_j$ , or of the source node of  $x_j$  if  $x_j$  is an edge. The delivered, wasted, and converted quantities are  $I D(\pi)$ ,  $I W(\pi)$ , and  $I C(\pi)$ . The conversion shortfall is reported and is not priced. The two estimates answer different questions and in general give different values:  $L_{\text{cap}}$  charges every component at capacity, including components that a given path does not use, while  $L_{\text{path}}$  charges only the components on  $\pi$  and only for what reaches them.

The library also computes a total cost of ownership over  $Y$  years at a discount rate  $d$ ,

$$\text{TCO} = K + J + \sum_{y=1}^Y \frac{O}{(1+d)^y} - \frac{S}{(1+d)^Y}, \quad (13)$$

where  $K$ ,  $J$ ,  $O$ , and  $S$  are the capital, installation, annual operating, and salvage values summed over the nodes.

## 4 The 1.x modules and associational analysis

The modules of version 1.x remain in the subpackage `ecoanalyst.legacy` so that existing scripts keep working. This section describes their network model and their regression module.

### 4.1 Typed nodes and loss functions

The class `AdvancedWasteNetwork` represents actors with typed classes (initial producer, food processor, food handler, end consumer, and solution provider) joined by inventory, service, and currency edges. It rejects inventory edges into an initial producer and inventory edges out of an end consumer. Each

node and edge has a loss function that is evaluated on the current conditions:

$$\text{static: } w_s = \alpha, \quad (14)$$

$$\text{time-based: } w_t(t) = \beta_0 + \beta_1 t, \quad (15)$$

$$\text{multivariable: } w_m(\mathbf{x}) = f(\mathbf{x}), \quad (16)$$

$$\text{regression-based: } w_r(\mathbf{x}) = \min(1, \max(0, \hat{\alpha} + \mathbf{x}^\top \hat{\boldsymbol{\beta}})), \quad (17)$$

where  $t$  is a time in the units of  $\beta_1$ ,  $f$  is a user function, and  $(\hat{\alpha}, \hat{\boldsymbol{\beta}})$  are posterior mean coefficients from Section 4.2. The path computation evaluates the functions at  $t = 24$  unless the caller gives another time, clamps each loss to  $[0, 1]$ , uses the inventory edge with the lowest loss between consecutive nodes, and compounds the losses as in Lemma 1 without conversion. Its routing uses the weights  $-\ln(1 - w_e)$  on edge losses only. Its arguments for a custom cost function and for capacity constraints are accepted and not used.

Solution providers and service edges store an effect multiplier  $\alpha_s \in (0, 1]$ , and processors store transformation rules for output composition. No loss calculation applies either of them. In version 3.0 the effect of a service on a loss rate is expressed in the causal graph of Section 5, where a variable for the service is a parent of the variable bound to the loss rate.

## 4.2 Bayesian regression of loss on its drivers

The class `WasteCausalNetwork` keeps its 1.x name for compatibility. For a loss column  $y$  and driver columns  $\mathbf{x}$  it fits the linear model

$$y_i \sim \mathcal{N}(\alpha + \mathbf{x}_i^\top \boldsymbol{\beta}, \sigma^2), \\ \alpha \sim \mathcal{N}(0, 10^2), \quad \beta_j \sim \mathcal{N}(0, 2^2), \quad \sigma \sim \text{Half-Normal}(1),$$

and, for a binary outcome, the logistic model  $y_i \sim \text{Bernoulli}(\text{logit}^{-1}(\alpha + \mathbf{x}_i^\top \boldsymbol{\beta}))$  with the same priors on  $\alpha$  and  $\boldsymbol{\beta}$ . The posterior is sampled by Markov chain Monte Carlo in PyMC [8]. The method `fit_regression` standardizes each driver to zero mean and unit variance, so its coefficients are per standard deviation of the driver. The method `fit` keeps the drivers on their original scale, so that the regression-based loss function (17) can apply the coefficients to raw values.

The two methods report different intervals. The method `fit_regression` reports, for each record, the central 95% interval of the posterior draws of  $\alpha + \mathbf{x}_i^\top \boldsymbol{\beta}$ . This interval describes uncertainty about the mean loss and excludes

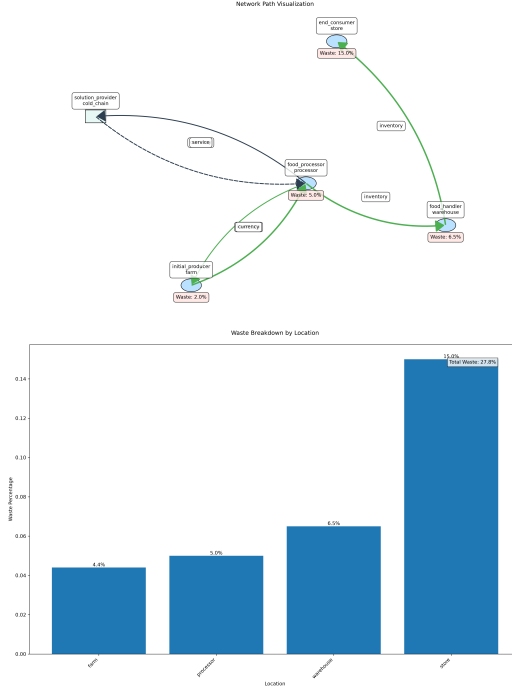


Figure 2: Drawing produced by the 1.x drawing helpers (`examples/network_visualization.py`) for the path farm, processor, warehouse, store. The farm uses the time-based loss (15) with 2% plus 0.1% per hour, evaluated at 24 hours. The processor and the store use static losses, and the warehouse uses a function of temperature and humidity. The node labels show each component’s base rate, so the farm reads 2.0% in the graph and 4.4% in the bar chart. The total printed in the chart, 27.8%, compounds the four evaluated rates as in Lemma 1.

the noise  $\sigma$ . The method `fit` reports  $\hat{y}_i \pm 1.96 s$ , where  $\hat{y}_i$  uses the posterior mean coefficients and  $s$  is the standard deviation of the residuals.

### 4.3 Interpretation

The posterior of  $\beta_j$  summarizes how the expected loss differs between records that differ in  $x_j$  and agree on the other included drivers. This is a property of the joint distribution that generated the data. It equals the effect of changing  $x_j$  by intervention only if two further conditions hold: the other included drivers block every back-door path from  $x_j$  to the loss (Section 5.7), and the

linear form is correct. The module checks neither condition, so its output is associational.

Two methods keep 1.x names and do not estimate causal quantities. The method `get_causal_effect` returns the product of user-supplied edge weights along the shortest path between two variables, together with the bootstrap standard deviation of a Pearson correlation. The method `discover_causal_structure` joins two columns when their marginal Pearson correlation is significant at a chosen level, and it orients each edge from the column name that sorts later to the one that sorts earlier.

#### 4.4 Example with simulated drivers

The script `examples/toy_ecosystem.py` simulates 1000 records with three independent drivers, temperature  $T \sim \mathcal{N}(20, 5^2)$ , humidity  $H \sim \mathcal{N}(50, 10^2)$ , and storage time  $S \sim \mathcal{N}(24, 6^2)$ . An unobserved damage variable is  $U = 0.3T + 0.2H + 0.4S + \varepsilon_1$ , and the recorded waste is  $Y = 0.8U + \varepsilon_2$ , with  $\varepsilon_1, \varepsilon_2 \sim \mathcal{N}(0, 1)$ . The implied slopes of  $\mathbb{E}[Y \mid T, H, S]$  are 0.24, 0.16, and 0.32. The regression of  $Y$  on  $(T, H, S)$  with `fit` gives posterior means 0.231, 0.153, and 0.329, with posterior standard deviations 0.009, 0.004, and 0.007, and  $R^2 = 0.812$ . Figure 3 shows the variable graph that the script builds, with the simulation coefficients as edge weights.

The agreement between the estimates and the simulated weights follows from the design of the simulation. The drivers are drawn independently, and no variable affects both a driver and the waste, so the conditional expectation coincides with the interventional one. In records collected during operation, a common cause such as the season can affect both storage time and temperature. The regression coefficients then do not have this interpretation unless the adjustment conditions of Section 5.7 hold.

### 5 A causal graph in parallel to the flow graph

The flow network answers questions of the form “given these loss rates, how much arrives”. The module `ecoanalyst.causal` adds a second graph that answers “why do the loss rates take these values, and how do they change under an intervention”. The two graphs stay separate. They are connected only through bindings from causal variables to parameters of the flow network.

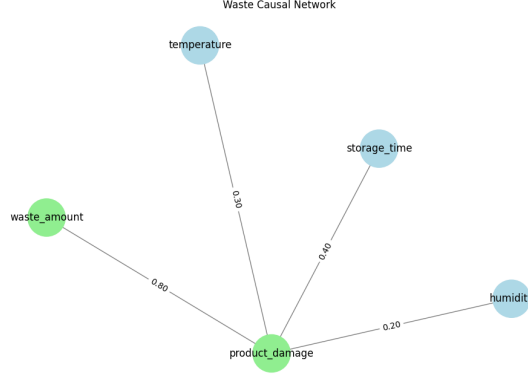


Figure 3: Variable graph drawn by the 1.x regression module for the simulated data of Section 4.4. The edge labels are the weights passed to `add_edge`, which are the coefficients used to simulate the data. The module stores these weights and does not estimate them. The graph records the association structure assumed between storage conditions and waste. The title inside the image is the name of the 1.x class.

## 5.1 Two graphs and a binding map

**Definition 4** (Causal graph). A causal graph is a directed acyclic graph (DAG)  $G_c = (V, A)$  over a finite set of variables  $V$ . Each variable  $X \in V$  has a kind  $k_X \in \{\text{continuous, rate, positive, binary}\}$ , a parent set  $\text{Pa}(X) = \{P : (P, X) \in A\}$ , and a flag that marks it as observed or unobserved.

Variables are added parents first. The insertion order is therefore a topological order, and the graph is acyclic by construction.

**Definition 5** (Binding map). A binding map is an injective map  $\phi$  from a subset  $V_B \subseteq V$  to coordinates of  $\theta$ . The admissible coordinates are the node waste rate, efficiency, and degradation rate, which require a rate variable, the node capacity, which requires a positive variable, the edge waste rate and efficiency, which require a rate variable, and the edge maximum rate, which requires a positive variable. For values  $V = \mathbf{v}$ , the bound parameter vector  $\theta(\mathbf{v})$  equals  $\theta$  except that coordinate  $\phi(X)$  takes the value  $v_X$  for every  $X \in V_B$ . The bound network is  $G_p[\theta(\mathbf{v})]$ .

The library refuses to simulate when a binding names a node or edge that is not in the network, or when two variables drive the same coordinate. A bound coordinate affects an outcome only if the outcome reads it. Capacity and maximum rate enter only the capacity-based cost (11). An efficiency matters only in the conversion and retention modes. No calculation in version 3.0 reads the degradation rate. Figure 4 in Section 6 shows a causal graph with two bindings.

## 5.2 Structural equations

Each variable  $X$  has a linear predictor on its link scale,

$$\eta_X = \beta_{X,0} + \sum_{P \in \text{Pa}(X)} \beta_{X,P} P, \quad (18)$$

in which the parents enter on their natural scale. With  $\varepsilon_X \sim \mathcal{N}(0, 1)$ ,  $U_X \sim \text{Uniform}(0, 1)$ , and a noise scale  $s_X \geq 0$ , the structural equation of  $X$  depends on its kind as shown in Table 1. A rate variable is therefore logit-normal given its parents, and a positive variable is log-normal given its parents. The coefficients  $\beta_X = (\beta_{X,0}, (\beta_{X,P})_{P \in \text{Pa}(X)})$  and the scale  $s_X$  are either stated by the user or drawn from the posterior of Section 5.6. A stated intercept is given on the link scale. For example, a rate variable with no parents and the intercept  $\text{logit}(0.05)$  has a median of 5%.

| Kind       | Link $h$ | Values        | Structural equation                                 |
|------------|----------|---------------|-----------------------------------------------------|
| continuous | identity | $\mathbb{R}$  | $X = \eta_X + s_X \varepsilon_X$                    |
| rate       | logit    | $(0, 1)$      | $X = \text{logit}^{-1}(\eta_X + s_X \varepsilon_X)$ |
| positive   | log      | $(0, \infty)$ | $X = \exp(\eta_X + s_X \varepsilon_X)$              |
| binary     | logit    | $\{0, 1\}$    | $X = \mathbf{1}\{U_X < \text{logit}^{-1}(\eta_X)\}$ |

Table 1: Variable kinds, link functions, and structural equations. Here  $\text{logit}^{-1}(z) = 1/(1 + e^{-z})$ , and the exogenous terms  $\varepsilon_X$  and  $U_X$  are independent across variables.

## 5.3 Outcomes

An outcome is computed for each joint sample. A flow outcome is a function  $g$  of the parameter vector, evaluated on the bound network:  $Y = g(G_p[\theta(V)])$ . The library provides the following flow outcomes:

- the waste fraction  $W(\pi)$  and the delivered fraction  $D(\pi)$  of a fixed path  $\pi$ ;
- the delivered fraction  $D(\pi^*(\theta))$  of the best route, where  $\pi^*(\theta)$  is the route of Proposition 2, recomputed for every parameter vector (the value is 0 when no route exists);
- the path loss cost  $L_{\text{path}}(\pi)$  and the capacity-based cost  $L_{\text{cap}}$ .

An outcome can also be the value of a causal variable. The flow outcome is evaluated separately for each sample, so  $g$  can be any function of  $\theta$ . The best-route outcome, for example, is continuous but not differentiable at parameter values where the best route changes.

## 5.4 Sampling and interventions

Ancestral sampling draws  $n$  joint samples. For each sample, the sampler visits the variables in topological order and computes each variable from the values already drawn for its parents and from fresh exogenous terms. The Monte Carlo estimate of the expected outcome is

$$\widehat{\mathbb{E}}[g] = \frac{1}{n} \sum_{i=1}^n g(G_p[\theta(\mathbf{v}^{(i)})]). \quad (19)$$

The library sets the bound parameters for each sample, evaluates the outcome, and restores the original values afterwards.

An intervention  $do(X = x)$  replaces the structural equation of  $X$  with the constant  $x$ . In the graph this deletes every arrow into  $X$ , which is the graph surgery of Pearl [7]. The result is the interventional distribution  $P(V \mid do(X = x))$ . Several variables can be set at once. The value is checked against the kind of the variable: a rate must lie in  $[0, 1]$ , a positive value must be greater than zero, and a binary value must be 0 or 1. An intervention on a bound variable fixes the corresponding network parameter. An intervention on an upstream variable changes the bound parameters through every directed path from that variable.

## 5.5 Effect estimation with shared exogenous noise

For two sets of interventions  $a$  and  $b$ , the effect on the outcome  $g$  is

$$\tau(a, b) = \mathbb{E}[g(G_p[\theta(V)]) \mid do(a)] - \mathbb{E}[g(G_p[\theta(V)]) \mid do(b)]. \quad (20)$$

The set  $b$  may be empty, and then the second term is the expectation under the unmodified model.

The function `compare` estimates  $\tau(a, b)$  with common random numbers (Algorithm 1). For every variable, and in the same order, the sampler draws the same random numbers whether or not the variable is set by an intervention. With the same seed, sample  $i$  under  $a$  and sample  $i$  under  $b$  therefore share their exogenous terms and their parameter draws, and they differ only through the interventions. The estimate is

$$\hat{\tau} = \frac{1}{n} \sum_{i=1}^n \Delta_i, \quad \Delta_i = g_a^{(i)} - g_b^{(i)}, \quad (21)$$

with variance  $\text{Var}(\hat{\tau}) = (\text{Var}(g_a) + \text{Var}(g_b) - 2 \text{Cov}(g_a, g_b))/n$ . Independent draws for the two regimes would make the covariance zero. When  $g_a$  and  $g_b$  respond in the same direction to the shared terms, the covariance is non-negative, and the variance of  $\hat{\tau}$  is no larger than with independent draws. The library reports  $\hat{\tau}$ , the 5th, 50th, and 95th percentiles of  $\Delta_i$ , and the share of samples with  $\Delta_i > 0$ . The percentiles describe how the effect varies across samples, that is, across exogenous conditions and parameter draws. They are not a credible interval for  $\tau$ .

When the posterior of an equation is used, each sample draws its own coefficients (Section 5.6). The average (21) then estimates the posterior mean of  $\tau$ , and the spread of  $\Delta_i$  combines parameter uncertainty with exogenous variation. With parameter uncertainty switched off, every sample uses the posterior mean.

## 5.6 Bayesian update

The function `fit` updates the equation of a variable  $X$  when the data contain a column for  $X$  and for each of its parents. It skips rows with a missing or non-finite value in any of these columns, and it leaves the other variables with their stated equations. Let  $Z$  be the  $n \times p$  design matrix with a column of ones and one column per parent, and let  $\mathbf{y} = h(\mathbf{x})$  be the observed values of  $X$  on the link scale. Rate values are clipped to  $[10^{-9}, 1 - 10^{-9}]$  before the logit.

---

**Algorithm 1** Estimate of  $\tau(a, b)$  with shared exogenous noise

---

**Require:** causal graph  $G_c$  with equations, flow network  $G_p$  with parameters  $\theta$ , binding map  $\phi$ , outcome  $g$ , intervention sets  $a$  and  $b$ , sample size  $n$ , seed

```

1: for each regime  $\rho \in \{a, b\}$  do
2:   start the random number generator from the seed
3:   for each variable  $X$  in topological order do
4:     draw, for all  $n$  samples, the coefficient normals,  $\varepsilon_X$ ,  $U_X$ , and (for a
       fitted normal-inverse-gamma posterior) a gamma variate for  $s_X^2$ 
5:     if  $\rho$  sets  $X$  to  $x$  then
6:        $X^{(i)} \leftarrow x$  for all  $i$ 
7:     else
8:       compute  $X^{(i)}$  from its equation, the parent values  $\text{Pa}(X)^{(i)}$ , and
       the draws
9:     end if
10:  end for
11:   $g_\rho^{(i)} \leftarrow g(G_p[\theta(\mathbf{v}^{(i)})])$  for  $i = 1, \dots, n$ 
12: end for
13:  $\Delta_i \leftarrow g_a^{(i)} - g_b^{(i)}$ 
14: return the mean of  $\Delta$ , its 5th, 50th, and 95th percentiles, and the share
    of  $\Delta_i > 0$ 

```

---

**Continuous, rate, and positive variables.** The prior is normal-inverse-gamma,

$$\boldsymbol{\beta} \mid s^2 \sim \mathcal{N}(\mathbf{m}_0, s^2 \lambda^2 I_p), \quad s^2 \sim \text{Inv-Gamma}(a_0, b_0), \quad (22)$$

with  $a_0 = 2$  and  $b_0 = s_0^2$ . Here  $\mathbf{m}_0$  holds the stated intercept and coefficients,  $s_0$  is the stated noise scale, and  $\lambda$  is a prior scale with default value 10. Because  $\mathbb{E}[s^2] = b_0/(a_0 - 1) = s_0^2$ , the stated noise scale sets the prior mean of the noise variance, and the prior standard deviation of each coefficient is  $\lambda$  times the noise scale. The posterior is normal-inverse-gamma with [4]

$$\Lambda_n = \lambda^{-2} I_p + Z^\top Z, \quad \mathbf{m}_n = \Lambda_n^{-1} (\lambda^{-2} \mathbf{m}_0 + Z^\top \mathbf{y}), \quad (23)$$

$$a_n = a_0 + \frac{n}{2}, \quad b_n = b_0 + \frac{1}{2} (\mathbf{y}^\top \mathbf{y} + \lambda^{-2} \mathbf{m}_0^\top \mathbf{m}_0 - \mathbf{m}_n^\top \Lambda_n \mathbf{m}_n). \quad (24)$$

This posterior is exact. To draw the parameters for one sample, the library computes

$$\begin{aligned} s^2 &= b_n/G, & G &\sim \text{Gamma}(a_n, 1), \\ \boldsymbol{\beta} &= \mathbf{m}_n + s L\mathbf{z}, & LL^\top &= \Lambda_n^{-1}, \quad \mathbf{z} \sim \mathcal{N}(0, I_p), \end{aligned}$$

so that  $s^2 \sim \text{Inv-Gamma}(a_n, b_n)$ . Without parameter uncertainty it uses  $\mathbf{m}_n$  and the noise scale  $\sqrt{b_n/(a_n - 1)}$ .

**Binary variables.** The prior is  $\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{m}_0, \lambda^2 I_p)$ . Newton’s method finds the posterior mode  $\hat{\boldsymbol{\beta}}$  with the gradient and Hessian of the negative log posterior  $\ell$ ,

$$\begin{aligned} \nabla \ell(\boldsymbol{\beta}) &= Z^\top (\mathbf{p} - \mathbf{y}) + \lambda^{-2}(\boldsymbol{\beta} - \mathbf{m}_0), \\ \nabla^2 \ell(\boldsymbol{\beta}) &= Z^\top \text{diag}(\mathbf{p} \odot (1 - \mathbf{p}))Z + \lambda^{-2}I_p, \end{aligned} \tag{25}$$

where  $\mathbf{p} = \text{logit}^{-1}(Z\boldsymbol{\beta})$ . It stops after 100 iterations or when the largest step is below  $10^{-10}$ . The Laplace approximation to the posterior is  $\boldsymbol{\beta} \mid \text{data} \approx \mathcal{N}(\hat{\boldsymbol{\beta}}, [\nabla^2 \ell(\hat{\boldsymbol{\beta}})]^{-1})$  [4].

When all variables are observed, the exogenous terms are independent, and the priors of different equations are independent, the likelihood of the data factorizes into one factor per variable,  $\prod_{X \in V} p(x \mid \text{Pa}(X), \boldsymbol{\beta}_X, s_X)$ . Fitting each equation separately then gives the joint posterior. The library does not integrate over unobserved variables. An unobserved variable has no column in the data, so neither its own equation nor the equations of its children can be fitted, and those equations keep their stated values.

## 5.7 Identification

An effect computed by Algorithm 1 is the effect implied by the model. It is identified from observational data only under assumptions about the graph. Two results apply.

**Truncated factorization.** Suppose that every variable is observed and that assumptions (A1), (A2), and (A5) of Section 5.8 hold. For an intervention  $do(X = x)$  and values  $\mathbf{v}$  with  $v_X = x$ , the interventional distribution is [7]

$$P(\mathbf{v} \mid do(X = x)) = \prod_{V_j \in V \setminus \{X\}} P(v_j \mid \text{pa}_j), \tag{26}$$

and it is zero for values with  $v_X \neq x$ . Here  $\text{pa}_j$  denotes the values of  $\text{Pa}(V_j)$  in  $\mathbf{v}$ . Each factor is a conditional distribution of observed variables, which `fit` estimates from observational data. Sampling with the equation of  $X$  replaced draws from this product. Under (A3) and (A4) for the support and form of each factor, the effect (20) is therefore identified.

**Back-door criterion.** When some variables are unobserved, the simulation relies on stated equations for their children. The library can still report whether an effect is estimable from observational data by adjustment.

**Definition 6** (Back-door criterion [6]). A set  $\mathbf{Z} \subseteq V \setminus \{X, Y\}$  satisfies the back-door criterion relative to  $(X, Y)$  if no element of  $\mathbf{Z}$  is a descendant of  $X$  and  $\mathbf{Z}$  blocks every path between  $X$  and  $Y$  that contains an arrow into  $X$ .

**Theorem 3** (Back-door adjustment [6, 7]). *If  $\mathbf{Z}$  satisfies the back-door criterion relative to  $(X, Y)$ , then  $P(y \mid \text{do}(X = x)) = \sum_{\mathbf{z}} P(y \mid x, \mathbf{z}) P(\mathbf{z})$ .*

The function `is_valid_adjustment( $X, Y, \mathbf{Z}$ )` returns true if and only if every element of  $\mathbf{Z}$  is observed,  $\mathbf{Z} \cap (\{X, Y\} \cup \text{De}(X)) = \emptyset$ , and  $\mathbf{Z}$  d-separates  $X$  and  $Y$  in the graph  $G_{\underline{X}}$  obtained from  $G_c$  by deleting the arrows out of  $X$ . The last condition is equivalent to blocking every back-door path. The function `adjustment_set( $X, Y$ )` returns a minimal set of observed non-descendants of  $X$  with this property, computed with the minimal d-separator routine of NetworkX. It returns the empty set when no adjustment is needed, and it returns `None` when no set of observed variables satisfies the criterion. The back-door criterion is sufficient and not necessary. When it fails, the effect can still be identifiable by other means, such as the front-door criterion, which the library does not check. The result tells a user which observed variables an estimate from data must condition on. That estimate can come from an outside regression or from DoWhy [9], which reads the graph that `to_gml` exports in GML.

## 5.8 Assumptions

The effects that the causal layer reports are causal effects only under the following assumptions. The library does not test them.

- (A1) *The DAG is correct.* Every direct effect among the modeled variables is an arrow in  $G_c$ , and every missing arrow is a claim of no direct effect.

- (A2) *No unmodeled common causes.* Every common cause of two modeled variables is itself in  $V$ . A common cause that is not measured must be included and marked unobserved.
- (A3) *Positivity.* The intervention values, and each stratum of an adjustment set, have support in the data. Outside the observed range the simulation extrapolates the linear predictor on the link scale.
- (A4) *Functional form.* Each equation is linear on its link scale in the natural-scale values of the parents, without interaction terms, with Gaussian noise on the link scale for continuous, rate, and positive variables and a logistic model for binary variables.
- (A5) *Independent exogenous noise.* The terms  $\varepsilon_X$  and  $U_X$  are independent across variables, and the rows used by `fit` are independent draws.

The structure of the flow network and its unbound parameters are held fixed. An intervention acts on the flow network only through the bound parameters.

## 6 Worked example: a produce cold chain

The script `examples/causal_cold_chain.py` builds the network and the causal graph of Figure 4. The numbers in this section come from that script, run with version 3.0.0 of the library, and from further calls on the network and model objects that it builds.

**Flow network.** A farm ( $w = 0.02$ ) ships produce to a retailer ( $w = 0.04$ ) on one of two routes. The first route passes a packhouse ( $w = 0.01$ ,  $\eta = 0.90$ ) and a cold store ( $w = 0.03$ ), with  $w = 0.01$  on each of its three edges. The second route is a direct unrefrigerated truck with  $w = 0.20$ . The network default mode is conversion, so the packhouse trims 10% of what it keeps. At these stated values the cold-chain route has

$$D = 0.98 \cdot 0.99 \cdot (0.99 \cdot 0.90) \cdot 0.99 \cdot 0.97 \cdot 0.99 \cdot 0.96 = 0.789,$$

$$W = 0.115, \quad C = 0.096,$$

and the truck route has  $D = 0.98 \cdot 0.80 \cdot 0.96 = 0.753$  and  $W = 0.247$ . Proposition 2 therefore selects the cold-chain route.

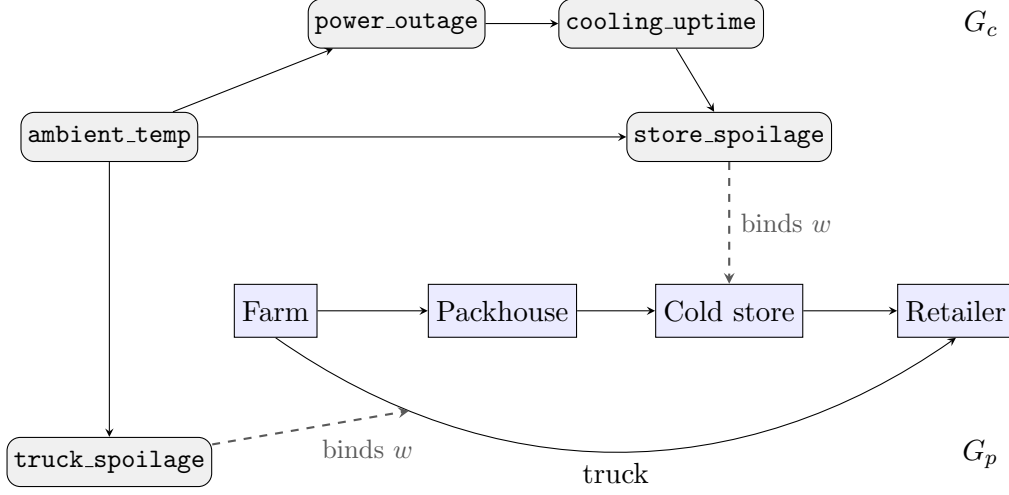


Figure 4: Causal graph  $G_c$  (rounded boxes) in parallel to the flow network  $G_p$  (square boxes) for the cold-chain example. Solid arrows between rounded boxes are causal arrows; solid arrows between square boxes are inventory edges. The dashed arrows are bindings: `store_spoilage` drives the waste rate of the cold store, and `truck_spoilage` drives the waste rate of the direct truck edge.

**Causal graph.** The graph has five variables: `ambient_temp` (continuous, in °C), `power_outage` (binary), `cooling_uptime` (rate), `store_spoilage` (rate, bound to the waste rate of the cold store), and `truck_spoilage` (rate, bound to the waste rate of the truck edge). The stated equations serve as the prior. Their intercepts are 24 for temperature,  $\text{logit}(0.95)$  for uptime,  $\text{logit}(0.03)$  for store spoilage, and  $\text{logit}(0.20)$  for truck spoilage, and their coefficients are zero.

**Data and fit.** The script generates 365 synthetic daily records from

$$\begin{aligned}
 T &\sim \mathcal{N}(24, 4^2), \\
 O &\sim \text{Bernoulli}(\text{logit}^{-1}(-9 + 0.25 T)), \\
 U_c &= \text{logit}^{-1}(3.5 - 3 O + 0.3 \varepsilon_1), \\
 S_s &= \text{logit}^{-1}(-3 + 0.08 T - 4 U_c + 0.2 \varepsilon_2), \\
 S_t &= \text{logit}^{-1}(-3.8 + 0.10 T + 0.2 \varepsilon_3),
 \end{aligned}$$

where  $T$ ,  $O$ ,  $U_c$ ,  $S_s$ , and  $S_t$  are temperature, outage, uptime, store spoilage, and truck spoilage, and  $\varepsilon_1, \varepsilon_2, \varepsilon_3 \sim \mathcal{N}(0, 1)$ . The records contain 20 outage days and 7 days at or above 32 °C. All five equations are fitted. Table 2 compares the posterior means with the generating values.

| Equation                    | Term                        | Generating value | Posterior mean |
|-----------------------------|-----------------------------|------------------|----------------|
| <code>ambient_temp</code>   | intercept                   | 24               | 23.964         |
|                             | noise scale                 | 4                | 3.762          |
| <code>power_outage</code>   | intercept                   | −9.0             | −8.402         |
|                             | <code>ambient_temp</code>   | 0.25             | 0.219          |
| <code>cooling_uptime</code> | intercept                   | 3.5              | 3.502          |
|                             | <code>power_outage</code>   | −3.0             | −3.052         |
|                             | noise scale                 | 0.3              | 0.306          |
| <code>store_spoilage</code> | intercept                   | −3.0             | −2.821         |
|                             | <code>ambient_temp</code>   | 0.08             | 0.073          |
|                             | <code>cooling_uptime</code> | −4.0             | −4.021         |
|                             | noise scale                 | 0.2              | 0.220          |
| <code>truck_spoilage</code> | intercept                   | −3.8             | −3.891         |
|                             | <code>ambient_temp</code>   | 0.10             | 0.103          |
|                             | noise scale                 | 0.2              | 0.216          |

Table 2: Fitted equations of the cold-chain example on the link scale (365 records). The noise scale column shows  $\sqrt{b_n/(a_n - 1)}$ . The binary equation for `power_outage` uses the Laplace approximation and has no noise scale.

**Identification.** For the effect of `cooling_uptime` on `store_spoilage`, the only back-door path is

$$\text{cooling\_uptime} \leftarrow \text{power\_outage} \leftarrow \text{ambient\_temp} \rightarrow \text{store\_spoilage}.$$

The function `adjustment_set` returns the set `{power_outage}`. The set `{ambient_temp}` also satisfies the criterion, and the empty set does not. For the effect of `power_outage` on `store_spoilage` the function returns `{ambient_temp}`, and for the effect of `ambient_temp` it returns the empty set. All five variables are observed, so the truncated factorization (26) also applies.

**Interventions.** Table 3 lists the results for  $n = 4000$  samples with seed 1 and parameter uncertainty switched on. The outcome for the first two rows is the delivered fraction of the best route,  $D(\pi^*(\theta))$ , with the route recomputed for every sample. The outcome for the third row is the capacity-based cost  $L_{\text{cap}}$  with prices 1.2 for the farm class, 2.0 for the cold store class, 3.0 for the retailer class, and the default price 1.0 for the packhouse class. The edges have no stated maximum rate and use the library default of 100 units per day, while each node has a capacity of 1000 kg per day.

| Quantity                                                        | Mean   | Percentiles |        |        | Share<br>> 0 |
|-----------------------------------------------------------------|--------|-------------|--------|--------|--------------|
|                                                                 |        | 5th         | 50th   | 95th   |              |
| $D(\pi^*)$ , no intervention                                    | 0.811  | 0.797       | 0.808  | 0.844  |              |
| $\Delta_i$ for $D(\pi^*)$ : $do(O = 0)$ vs. none                | 0.0014 | 0.0000      | 0.0000 | 0.0104 | 5.6%         |
| $\Delta_i$ for $L_{\text{cap}}$ : $do(T = 32)$ vs. $do(T = 24)$ | 45.6   | 25.6        | 31.4   | 124.2  | 100%         |

Table 3: Simulation results for the cold-chain example ( $n = 4000$ , seed 1). The first row describes the delivered fraction of the best route without intervention. The other rows describe the per-sample differences  $\Delta_i$ , whose mean is the effect estimate  $\hat{\tau}$  of (21). Under  $do(T = 32)$  and  $do(T = 24)$  the mean costs are 243.9 and 198.3.

Without intervention, the median delivered fraction is 0.808, above the stated-parameter value 0.789. The data move the loss rate of the cold store from the stated 3% to a median of 0.7% across samples. In 19.9% of the samples the router chooses the direct truck. These samples have a mean temperature of 19.5 °C, the truck loses less on such days, and they form the upper tail of the delivered fraction.

Backup power,  $do(O = 0)$ , raises the delivered fraction by 0.0014 on average. The effect is small because outages are rare: the fitted outage probability at 24 °C is  $\text{logit}^{-1}(-8.402 + 0.219 \cdot 24) = 0.042$ . In most samples no outage occurs under the model, so  $\Delta_i = 0$ . The 5.6% of samples with  $\Delta_i > 0$  are samples with an outage in which the router uses the cold chain, and their mean difference is about 0.025. The mean alone does not show that the benefit is concentrated on a small share of the days.

A heat wave,  $do(T = 32)$  against  $do(T = 24)$ , raises the capacity-based cost by 45.6 per day. Temperature reaches the cost through three directed paths: directly to `store_spoilage`, through `power_outage` and `cooling_uptime` to `store_spoilage`, and to `truck_spoilage`. The right tail of  $\Delta_i$

(95th percentile 124.2 against a median of 31.4) comes from samples in which the higher temperature produces an outage that does not occur at 24 °C. Both interventions set values that occur in the records, so assumption (A3) holds for them.

## 7 Implementation

EcoAnalyst is a Python package for Python 3.10 or later. Its core depends on NetworkX [5], NumPy, and pydantic. Nodes and edges are validated pydantic models, mirrored into a NetworkX `MultiDiGraph`. Networks are read and written as JSON in a native format, a flow-graph exchange format, and the 1.x format, and a causal model is stored with its network. The same functions are available through a command-line tool, a local REST server, and a Model Context Protocol (MCP) server. The 1.x modules and the PyMC-based regression are optional extras. The numbers in Sections 4.4 and 6 come from example scripts in the repository and from the objects that those scripts build.

## 8 Discussion

The product form of path loss follows from the definition of a rate as a fraction of what reaches a component. Two consequences follow. The loss attributed to downstream components is smaller than their raw rates suggest, because less reaches them. Routing becomes a shortest-path problem only after the logarithmic transform, and the transform needs non-negative weights, which the constraint  $r \in (0, 1]$  provides.

The regression module and the causal layer can be fitted to the same records and still answer different questions. A regression coefficient describes the distribution of the data. An effect from `compare` describes how the model responds to an intervention, and it is a causal effect only under (A1) to (A5). The back-door check connects the two views: it states which regression, on which observed variables, estimates the same effect from data.

The model has the following limitations:

- Routing follows a fraction of one unit on a single path, and capacities do not constrain it.

- Parameters are static within a sample. The model has no dynamics, and the degradation rate is stored but not used.
- Structural equations are linear on the link scale and have no interaction terms.
- The fit does not integrate over unobserved variables, and the back-door criterion is the only identification check.
- The worked example uses synthetic data generated from a model in the same family as the fitted one. It demonstrates the procedure, not performance on field data.

## 9 Conclusion

This paper stated the model of EcoAnalyst 3.0 as the library implements it. Path loss is a product of retained fractions, and minimum-loss routing is a shortest-path problem under the weights  $-\ln r$ . Efficiency has three explicit semantics. The 1.x regression module gives associations between loss and its drivers. The causal layer binds a DAG of structural equations to the parameters of the flow network, so that an intervention on an upstream variable reaches any flow outcome through the bound parameters, under stated assumptions and with a back-door check. Future work includes capacity-constrained routing as a generalized flow problem, equations with interaction terms, identification criteria beyond back-door adjustment, and inference that integrates over unobserved variables.

## References

- [1] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ, 1993.
- [2] Chetna Chauhan, Amandeep Dhir, M Usman Akram, and Jari Salo. Food waste in supply chains: A systematic literature review and framework development. *Journal of Cleaner Production*, 295:126438, 2021.
- [3] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

- [4] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. Chapman and Hall/CRC, Boca Raton, FL, third edition, 2013.
- [5] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, Pasadena, CA, 2008.
- [6] Judea Pearl. Causal diagrams for empirical research. *Biometrika*, 82(4):669–688, 1995.
- [7] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, second edition, 2009.
- [8] John Salvatier, Thomas V Wiecki, and Christopher Fonnesbeck. Probabilistic programming in Python using PyMC3. *PeerJ Computer Science*, 2:e55, 2016.
- [9] Amit Sharma and Emre Kiciman. DoWhy: An end-to-end library for causal inference. arXiv preprint arXiv:2011.04216, 2020.
- [10] Jin Y. Yen. Finding the  $k$  shortest loopless paths in a network. *Management Science*, 17(11):712–716, 1971.